

16. Case Studies & Build Logs

This concluding section collects the book’s artifact-oriented case studies and build logs. Its purpose is not to introduce new theory, but to show how the preceding notions of closure, symmetry, modularity, and multi-resolution analysis behave in practice when one moves from abstract specifications to executable designs, optimization passes, and failure analysis.

A recurring theme is the use of multi-resolution dashboards, denoted here by $\varepsilon(r)$, to track a design across several levels of abstraction: function, gate, timing, and device. At the function level, one asks whether the intended Boolean behavior is preserved under rewriting, factoring, or decomposition. At the gate level, the concern shifts to structural cost, fan-in constraints, and local equivalence under identities. At the timing level, the same artifact is evaluated for critical path length, slack, and the stability of closure under retiming or pipelining. At the device level, the question becomes whether the implementation remains realizable under the physical constraints of the target technology, including area, power, and routing pressure. The point of $\varepsilon(r)$ is that a single scalar or pass/fail verdict is usually insufficient; closure must be monitored as a family of error or deviation measures indexed by resolution.

The first build log traces a progression from a NAND-only basis to a 4-bit ALU and then to a small microcontroller-style datapath. The NAND stage serves as the minimal functional seed: from a single universal gate, one synthesizes the standard Boolean primitives and verifies that the resulting library is closed under the intended composition rules. The next stage lifts this basis into a 4-bit arithmetic logic unit, where the design problem is no longer merely functional completeness but compositional regularity. Here, the relevant questions include whether adders, multiplexers, comparators, and control logic can be assembled from a shared set of subcircuits without violating interface contracts, and whether the resulting hierarchy admits clean reuse of verified components. The final stage extends the same methodology to a microcontroller-like system, where the build log emphasizes integration rather than isolated correctness: instruction decode, register transfer, control flow, and memory-facing interfaces must all remain consistent under optimization and refinement.

Across these case studies, optimization is treated as a sequence of semantics-preserving transformations rather than as an ad hoc search for smaller netlists. One important family of transformations is compression by symmetry, especially LUT packing and factoring. If a circuit contains repeated or permuted substructures, then symmetry-aware factoring can identify common subexpressions, reduce duplication, and pack equivalent logic into fewer lookup tables or fewer structural blocks. In practice, this means that the build log records not only the final size of the artifact but also the symmetry classes exploited during optimization, the factoring choices made at each step, and the extent to which the transformed design remains equivalent to the source specification. The relevant metric is not just gate count, but the tradeoff between structural compression and the preservation of closure under the chosen equivalence relation.

A useful way to read these logs is as a sequence of checkpoints. At each checkpoint, the artifact is compared against the current specification at the appropriate resolution, and the result is recorded together with the transformation history. For example, a function-level equivalence certificate may be paired with a gate-level resource report and a timing report that exposes a newly critical path. This layered record makes it possible to distinguish genuine improvement from merely shifted cost: a rewrite that reduces area but destroys timing closure is not a success in the operational sense, even if it is algebraically valid. Likewise, a retiming pass that preserves behavior but obscures the structure needed for later verification may be acceptable only if the build protocol explicitly allows that loss of transparency.

The post-mortem component of the section records closure failures as counterexamples. These

failures are valuable because they identify where a proposed hierarchy, rewrite system, or optimization pipeline does not actually close under composition, refinement, or implementation constraints. A closure failure may arise when a local rewrite is valid in isolation but breaks a global invariant; when a factoring step introduces an interface mismatch; when a timing optimization preserves functionality but violates the target clock budget; or when a symmetry reduction collapses distinctions that are operationally significant. Each counterexample is therefore documented with enough context to reproduce the failure, diagnose the violated assumption, and classify the failure mode relative to the book’s earlier closure notions.

In the most informative failures, the counterexample is not merely a bug report but a boundary marker for the theory. It shows where a closure claim must be weakened, where an equivalence relation must be refined, or where an optimization pass must be constrained by additional contracts. For instance, a transformation that is sound at the Boolean level may fail to respect a device-level constraint such as fanout, placement, or routing congestion. Conversely, a design that is physically realizable may fail to admit a clean algebraic factorization because the chosen decomposition destroys a symmetry that the later stages depend on. The logs therefore serve as a bridge between theorem and engineering practice: they show how abstract closure properties are tested, qualified, and sometimes rejected in the presence of real implementation constraints.

The artifact discipline is central to these logs. Each case study is accompanied by a repository index, schema definitions, and toolchain specifications so that the build can be replayed and audited. The repository index identifies the source artifacts, generated outputs, and intermediate checkpoints. The schemas specify the expected structure of netlists, timing reports, equivalence certificates, and optimization traces. The toolchain specification records the synthesis, verification, and analysis tools used, together with version constraints and invocation parameters. This makes the section not merely descriptive but reproducible: the reader can inspect the same pipeline that produced the reported results, compare alternative runs, and extend the open problems list with new experiments.

The open problems at the end of the section are intentionally practical. They include questions about how to define robust multi-resolution error metrics that remain meaningful across abstraction layers; how to automate symmetry-aware compression without obscuring debugging information; how to characterize the boundary between successful closure and acceptable approximation in large designs; and how to standardize build logs so that optimization histories can be compared across toolchains and technologies. In this sense, the section closes the book by returning from theorem and construction to the engineering reality of incomplete closure, partial success, and reproducible failure.

Artifact index. The accompanying repository index, schemas, and toolchain specifications are intended as living documents. They should record the exact build inputs, the transformation sequence, the verification checkpoints, and the unresolved issues associated with each case study. Readers are encouraged to treat these artifacts as templates for their own closure experiments, extending them with additional resolutions, alternative optimization passes, and new counterexamples as the theory and tooling evolve.

Build-log template. A minimal record for each experiment should include: the source specification; the target resolution(s) r used in the dashboard $\varepsilon(r)$; the sequence of rewrites, decompositions, and optimizations; the equivalence relation used for validation; the timing and device constraints, if applicable; and the final status, including any unresolved discrepancies. When a run fails, the log should preserve the smallest counterexample known, the stage at which closure was lost, and

the hypothesis that best explains the failure.

Reproducibility note. The section is designed to be replayable from the repository index and schema files alone. Any future extension should preserve this property by keeping generated artifacts, intermediate checkpoints, and toolchain versions synchronized with the narrative description of the build.

Suggested reading of the logs. Read the case studies as a ladder of resolutions: first the functional specification, then the structural realization, then the timing and device constraints, and finally the failure modes that reveal the limits of the chosen abstraction. The value of the logs lies not only in successful closure, but in the explicit recording of where closure stops being available.